

IAM, SECRETS, CONFIGURATION AND ENVIRONMENT BOUNDARIES

Lab Manual

Three labs and two activities on the Bookshelf services you already deployed — Cloud Shell and the console only, nothing to code.

Version 1.0 · September 2026 · Cloud Shell gcloud 583.x · region asia-south1

Contents

How to use this manual.....	3
Activity A — Find the over-grants.....	4
Lab 1 — An identity with nothing, a token you can read, and actAs.....	5
Lab 2 — A managed secret, mounted and injected, then rotated.....	9
Activity B — Boundaries for Bookshelf.....	13
Lab 3 — Least privilege, managed secret, internal ingress, two audit answers.....	14
Appendix — Environment, versions and the kit.....	17

How to use this manual

The slides teach the model and point to a lab; this manual has every step. All three labs run on bookshelf-api from the Cloud Run session, which should still be deployed in your sandbox project. Nothing is coded: the only 'new' software is bookshelf-api 1.1, supplied complete in the kit, which adds an endpoint that reports whether a secret reached the container — never the value.

Conventions

- Cloud Shell blocks (teal) are copy-pasteable. Run them one block at a time, in order; several steps sleep 30 s on purpose for IAM to propagate.
- Expected panels show the real output format. Project ids, numbers, hashes and timestamps differ in your project; the shape is what matters.
- Console path lines give the equivalent clicks where one exists.
- Variables PROJECT, PN, R, ME, AR and API_URL are set once at the top of labs.sh and reused — if you open a new Cloud Shell tab, run the first block again.
- Lab 3 ends WITHOUT a teardown. The CI/CD and Incident sessions start from the state it leaves.

Getting the kit into Cloud Shell

Upload gcp-iam-labs.zip through the Cloud Shell menu (⋮ → Upload) or drag it onto the terminal, then:

```
$ unzip -q gcp-iam-labs.zip && cd gcp-iam-labs && ls
README.md  activity-a-policy.yaml  activity-b-template.md  bookshelf-api  labs.sh
prep-project.sh  teardown.sh
$ PROJECT=$(gcloud config get-value project); PN=$(gcloud projects describe $PROJECT
--format='value(projectNumber)')
$ R=asia-south1; ME=$(gcloud config get-value account);
AR=$R-docker.pkg.dev/$PROJECT/bookshelf
$ gcloud run services list --region $R --format='value(SERVICE)'
bookshelf-api
bookshelf-web
```

If the last command prints nothing, bookshelf-api is gone: run the commented rebuild block at the top of labs.sh (about three minutes) before Lab 1. Only bookshelf-api and the two service accounts bookshelf-api-sa / bookshelf-web-sa are needed today.

What is already true when you start

Resource	State from the Cloud Run session
bookshelf-api	deployed, authenticated-only, runs as bookshelf-api-sa
bookshelf-web	deployed, public, runs as bookshelf-web-sa, calls the api with an identity token
IAM on bookshelf-api	roles/run.invoker → bookshelf-web-sa (on the service, not the project)
Project policy	you as Owner; the two service accounts hold no project roles

Activity A — Find the over-grants

Time	Format	Material
10 min + debrief	pairs, paper	activity-a-policy.yaml in the kit (also printed below)

Below is the IAM policy of a fictional production project, orders-prod — twelve bindings. Three of them violate least privilege. For each one you find, write the binding number, the narrower role that should replace it, and the resource it should be granted on. One more binding is arguable; say why if you spot it.

THE POLICY

```
bindings:
  1 - members: [group:platform-admins@example.com]
    role: roles/owner
  2 - members: [group:orders-developers@example.com]
    role: roles/viewer
  3 - members: [user:ravi@example.com]
    role: roles/editor
  4 - members: [serviceAccount:orders-api-sa@orders-prod.iam.gserviceaccount.com]
    role: roles/editor
  5 - members: [serviceAccount:orders-web-sa@orders-prod.iam.gserviceaccount.com]
    role: roles/secretmanager.secretAccessor
  6 - members: [serviceAccount:orders-deploy@ci-pipeline.iam.gserviceaccount.com]
    role: roles/run.admin
  7 - members: [serviceAccount:orders-deploy@ci-pipeline.iam.gserviceaccount.com]
    role: roles/iam.serviceAccountUser
  8 - members: [group:orders-oncall@example.com]
    role: roles/run.viewer
  9 - members: [group:orders-oncall@example.com]
    role: roles/logging.viewer
  10 - members: [group:orders-oncall@example.com]
    role: roles/run.admin
    condition: { title: incident-window, expression: request.time < timestamp("2026-09-12T18:00:00Z") }
  11 - members: [serviceAccount:service-110919260780@serverless-robot-prod.iam.gserviceaccount.com]
    role: roles/run.serviceAgent
  12 - members: [group:finance@example.com]
    role: roles/billing.viewer
```

#	What is wrong	Replace with (role)	Granted on (resource)
arguable			

Read each binding as a sentence — WHO may do WHAT, WHERE, for how long — and ask two questions of it: is the role the narrowest that does the job, and is the scope the narrowest?

Lab 1 — An identity with nothing, a token you can read, and actAs

Estimated time	Difficulty	Environment	Kit
25 min	●●○	Cloud Shell, region asia-south1	gcp-iam-labs.zip (labs.sh + bookshelf-api/ 1.1 + activity files)

OBJECTIVE

bookshelf-api running on a brand-new service account with zero roles; an identity token minted for bookshelf-web-sa and decoded; a wrong-audience token refused; a deployer identity blocked by actAs and then allowed; and a list of every role each identity holds.

PREREQUISITES

- bookshelf-api deployed and authenticated-only (see 'How to use this manual').
- Kit unzipped; variables from the first block set.

Steps

Step 1 — Where we start — what the api runs as, and what roles that identity holds

CLOUD SHELL

```
gcloud run services describe bookshelf-api --region $R --
format='value(spec.template.spec.serviceAccountName)'
gcloud projects get-iam-policy $PROJECT --flatten='bindings[].members' --
filter='bindings.members:bookshelf-api-sa' --format='table(bindings.role)'
```

EXPECTED

```
bookshelf-api-sa@gcptraining-507817.iam.gserviceaccount.com
# second command prints nothing: no project-level
roles
```

If the second command lists roles, they were added outside the course — note them; they are over-grants.

Step 2 — A brand-new identity with no roles; run the api as it — it still serves

Console path: Cloud Run → bookshelf-api → Edit & deploy new revision → Security → Service account

CLOUD SHELL

```
gcloud iam service-accounts create bookshelf-api-min --display-name="bookshelf-api
least-privilege"
gcloud run services update bookshelf-api --region $R --service-account bookshelf-api-
min@$PROJECT.iam.gserviceaccount.com
API_URL=$(gcloud run services describe bookshelf-api --region $R --
format='value(status.url)')
curl -s -o /dev/null -w "anonymous: HTTP %{http_code}\n" $API_URL/
curl -s -w "\nme: HTTP %{http_code}\n" -H "Authorization: Bearer $(gcloud auth print-
identity-token)" $API_URL/
```

EXPECTED

```
anonymous: HTTP 403
{"books":4,"greeting":"Bookshelf catalogue","service":"bookshelf-api","version":"v1"}
me: HTTP 200
```

A service account needs no roles to serve HTTP. Roles are only needed when the service calls other Google APIs — which is exactly when you will see a permission error and add one.

Step 3 — Impersonation is a permission too — mint bookshelf-web-sa's token and read it

Minting a token for another identity needs Service Account Token Creator on that identity. Grant it to yourself, wait for IAM, then mint a token whose audience is the api URL and decode its payload.

CLOUD SHELL

```
gcloud iam service-accounts add-iam-policy-binding bookshelf-web-
sa@$PROJECT.iam.gserviceaccount.com \
  --member=user:$ME --role=roles/iam.serviceAccountTokenCreator
sleep 30
TOKEN=$(gcloud auth print-identity-token --impersonate-service-account=bookshelf-web-
sa@$PROJECT.iam.gserviceaccount.com --audiences=$API_URL --include-email)
echo $TOKEN | cut -d. -f2 | tr '_' '/' | base64 -d 2>/dev/null | python3 -m
json.tool
curl -s -o /dev/null -w "as web-sa: HTTP %{http_code}\n" -H "Authorization: Bearer
$TOKEN" $API_URL/
```

EXPECTED

```
{
  "aud": "https://bookshelf-api-110919260780.asia-south1.run.app",
  "azp": "1069...",
  "email": "bookshelf-web-sa@gcptraining-507817.iam.gserviceaccount.com",
  "email_verified": true,
  "exp": 1757411187,
  "iat": 1757407587,
  "iss": "https://accounts.google.com",
  "sub": "1069..."
}
as web-sa: HTTP 200          # the invoker binding from the Cloud Run session lets
web-sa in
```

exp minus iat is 3600 seconds: the token lives one hour. Nothing in it is secret; the signature (third part, not decoded) is what makes it trustworthy.

Step 4 — A wrong-audience token is refused before IAM is consulted**CLOUD SHELL**

```
BAD=$(gcloud auth print-identity-token --impersonate-service-account=bookshelf-web-
sa@$PROJECT.iam.gserviceaccount.com --audiences=https://example.com --include-email)
curl -s -o /dev/null -w "wrong audience: HTTP %{http_code}\n" -H "Authorization:
Bearer $BAD" $API_URL/
```

EXPECTED

```
wrong audience: HTTP 401
```

Same identity, same invoker binding — refused, because the token was minted for a different service. A stolen token for one service is useless against another.

Step 5 — actAs — a deployer that may deploy but may not run code as bookshelf-api-min

Create a deployer identity with Cloud Run Admin on the project, let yourself impersonate it, and try to update the service through it. Deploying a service that runs AS bookshelf-api-min needs actAs on bookshelf-api-min — which the deployer does not have.

CLOUD SHELL

```
gcloud iam service-accounts create bookshelf-deployer --display-name="deployer (no
actAs yet)"
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:bookshelf-
deployer@$PROJECT.iam.gserviceaccount.com --role=roles/run.admin
gcloud iam service-accounts add-iam-policy-binding bookshelf-
deployer@$PROJECT.iam.gserviceaccount.com --member=user:$ME
--role=roles/iam.serviceAccountTokenCreator
sleep 30
gcloud run services update bookshelf-api --region $R --update-env-vars
GREETING="deployed by the deployer" \
  --impersonate-service-account=bookshelf-deployer@$PROJECT.iam.gserviceaccount.com
```

EXPECTED

```
ERROR: (gcloud.run.services.update) PERMISSION_DENIED: Permission
'iam.serviceaccounts.actAs' denied on
service account bookshelf-api-min@gcptraining-507817.iam.gserviceaccount.com (or it
may not exist).
```

Cloud Run Admin lets the deployer change services; it does not let it run code under an identity it has no actAs on. That is the gate.

Step 6 — Grant Service Account User on the one service account, and retry

Console path: IAM & Admin → Service Accounts → bookshelf-api-min → Permissions → Grant access

CLOUD SHELL

```
gcloud iam service-accounts add-iam-policy-binding bookshelf-api-
min@$PROJECT.iam.gserviceaccount.com \
  --member=serviceAccount:bookshelf-deployer@$PROJECT.iam.gserviceaccount.com --
  role=roles/iam.serviceAccountUser
sleep 30
gcloud run services update bookshelf-api --region $R --update-env-vars
GREETING="deployed by the deployer" \
  --impersonate-service-account=bookshelf-deployer@$PROJECT.iam.gserviceaccount.com
curl -s -H "Authorization: Bearer $(gcloud auth print-identity-token)" $API_URL/ |
grep -o '"greeting": "[^"]*"'
```

EXPECTED

```
✓ Deploying... Done.
"greeting": "deployed by the deployer"
```

The grant is on bookshelf-api-min, not on the project: the deployer may act as this one identity and no other. Session D's pipeline identity gets exactly this.

Step 7 — The answer sheet — every role each identity holds**CLOUD SHELL**

```
for SA in bookshelf-api-min bookshelf-web-sa bookshelf-deployer; do
  echo "== $SA"; gcloud projects get-iam-policy $PROJECT --
  flatten='bindings[].members' --filter="bindings.members:$SA@" --
  format='value(bindings.role)'
done
gcloud run services get-iam-policy bookshelf-api --region $R --
format='value(bindings)'
gcloud iam service-accounts get-iam-policy bookshelf-api-
min@$PROJECT.iam.gserviceaccount.com --format='value(bindings)'
```

EXPECTED

```
== bookshelf-api-min
== bookshelf-web-sa
== bookshelf-deployer
roles/run.admin
{'members': ['serviceAccount:bookshelf-web-sa@...'], 'role': 'roles/run.invoker'}
{'members': ['serviceAccount:bookshelf-deployer@...'], 'role':
'roles/iam.serviceAccountUser'}
```

Identity	Project-level roles	Resource-level bindings	Why
bookshelf-api-min	none	none (yet — Lab 2 adds secretAccessor on one secret)	it serves HTTP; it calls no Google API
bookshelf-web-sa	none	run.invoker on bookshelf-api	it must call the api and nothing else

Identity	Project-level roles	Resource-level bindings	Why
bookshelf-deployer	run.admin	serviceAccountUser on bookshelf-api-min	it deploys the api and may act only as the api's identity

CHECK YOURSELF

- The api answers 200 to you and 403 to anonymous while running as an identity with no roles.
- You decoded a token and found aud, email and exp.
- The wrong-audience token got 401, not 403 — audience is checked before IAM.
- You saw the actAs error and fixed it with one binding on one service account.

Symptom	Cause	Fix
print-identity-token: PERMISSION_DENIED / getOpenIdToken	Token Creator not propagated yet	wait 30–60 s and retry
Step 5 succeeds instead of failing	you are Owner and forgot -- impersonate-service-account	add the flag — the point is to act as the deployer
Step 6 still fails	propagation	sleep 30 again; IAM on service accounts can take a minute

Lab 2 — A managed secret, mounted and injected, then rotated

Estimated time	Difficulty	Environment	Kit
30 min	●●●	Cloud Shell, region asia-south1	gcp-iam-labs.zip (labs.sh + bookshelf-api/ 1.1 + activity files)

OBJECTIVE

A secret in Secret Manager with access granted to bookshelf-api-min only; bookshelf-api 1.1 receiving it both as a mounted file and as an environment variable; a rotation to version 2 that the file follows and the env var does not; version 1 disabled and its consequence observed; a read by the wrong identity refused.

PREREQUISITES

- Lab 1 complete: bookshelf-api runs as bookshelf-api-min.
- The kit's bookshelf-api/ folder (version 1.1) — supplied complete, nothing to edit.

Steps

Step 1 — Build bookshelf-api 1.1 with Cloud Build (adds GET /config)

/config reports whether a secret reached the container as a file at /secrets/db-password or as the env var DB_PASSWORD — as a length and a SHA-256 prefix. The value is never returned or logged.

CLOUD SHELL

```
gcloud services enable secretmanager.googleapis.com
cat bookshelf-api/app.py | grep -A3 'def config'
(cd bookshelf-api && gcloud builds submit --tag $AR/bookshelf-api:1.1)
```

EXPECTED

```
def config():
    """Fingerprints of the secret as the container sees it – never the value."""
    ...
SUCCESS    asia-south1-docker.pkg.dev/gcptesting-507817/bookshelf/bookshelf-api:1.1
```

Step 2 — Create the secret with version 1 — from a file, never on the command line

Console path: Security → Secret Manager → Create secret

CLOUD SHELL

```
printf 'v1-correct-horse' > /tmp/db-password.txt
gcloud secrets create db-password --replication-policy=automatic --data-file=/tmp/db-password.txt
rm /tmp/db-password.txt
gcloud secrets versions list db-password
```

EXPECTED

```
Created secret [db-password].
Created version [1] of the secret [db-password].
NAME    STATE    CREATED                DESTROYED
1       enabled  2026-09-09T09:11:05    -
```

A value typed as a command argument is written to Cloud Shell's shell history in plain text. --data-file from a temporary file keeps it out.

Step 3 — Access control on the secret, for the api identity only

Console path: Secret Manager → db-password → Permissions → Grant access

CLOUD SHELL

```
gcloud secrets add-iam-policy-binding db-password \
  --member=serviceAccount:bookshelf-api-min@$PROJECT.iam.gserviceaccount.com --
  role=roles/secretmanager.secretAccessor
gcloud secrets get-iam-policy db-password
```

EXPECTED

```
bindings:
- members:
  - serviceAccount:bookshelf-api-min@gcptraining-507817.iam.gserviceaccount.com
  role: roles/secretmanager.secretAccessor
```

On the secret, not on the project. A project-level secretAccessor would cover every secret the project ever has.

Step 4 — Two revisions — the secret as a file tracking latest, then also as an env var pinned to 1

Console path: Cloud Run → bookshelf-api → Edit & deploy → Container → Variables & Secrets → Reference a secret (mounted as volume / exposed as environment variable)

CLOUD SHELL

```
gcloud run services update bookshelf-api --region $R --image $AR/bookshelf-api:1.1 \
  --update-secrets=/secrets/db-password=db-password:latest --tag file
gcloud run services update bookshelf-api --region $R --update-secrets=DB_PASSWORD=db-
password:1 --tag env
curl -s -H "Authorization: Bearer $(gcloud auth print-identity-token)" $API_URL/config
| python3 -m json.tool
```

EXPECTED

```
{
  "config_env": {"APP_VERSION": "v1", "GREETING": "deployed by the deployer"},
  "instance": "bookshelf-api-00007-xyz-1-abc",
  "secret_env": {"length": 16, "present": true, "sha256_8": "9b3c1e7a"},
  "secret_file": {"length": 16, "present": true, "sha256_8": "9b3c1e7a"}
}
```

Same fingerprint from both paths: version 1 either way. The revision stores references, not values — check with `gcloud run services describe ... --format='yaml(spec.template.spec.containers[0].env)'`.

Step 5 — Rotate — add version 2 and read again**CLOUD SHELL**

```
printf 'v2-battery-staple' > /tmp/db-password.txt
gcloud secrets versions add db-password --data-file=/tmp/db-password.txt; rm /tmp/db-
password.txt
sleep 10; curl -s -H "Authorization: Bearer $(gcloud auth print-identity-token)"
$API_URL/config | python3 -m json.tool
```

EXPECTED

```
"secret_env": {"length": 16, "present": true, "sha256_8": "9b3c1e7a"},      #
unchanged — resolved once at instance start
"secret_file": {"length": 17, "present": true, "sha256_8": "4e0d88f2"}    # new —
  fetched on every read
```

No redeploy happened. The mounted file follows 'latest'; the env var is whatever the instance was born with — and it is pinned to 1 anyway.

Step 6 — Disable version 1 — and see what a pinned env var does on the next instance**CLOUD SHELL**

```
gcloud secrets versions disable 1 --secret=db-password
gcloud secrets versions list db-password
gcloud run services update bookshelf-api --region $R --update-env-vars ROTATED=yes
```

EXPECTED

```
NAME    STATE    CREATED
2       enabled  2026-09-09T09:41:12
1       disabled 2026-09-09T09:11:05
ERROR: (gcloud.run.services.update) The user-provided container failed to start ...
Secret version
projects/110919260780/secrets/db-password/versions/1 is DISABLED ...
```

The running instance still holds version 1 in memory; a NEW instance cannot fetch a disabled version, so the new revision fails to become ready and traffic stays on the previous one. This is why 'disable before destroy': you found the stale consumer without losing the value.

Step 7 — Finish the rotation — pin the env var to version 2**CLOUD SHELL**

```
gcloud run services update bookshelf-api --region $R --update-secrets=DB_PASSWORD=db-
password:2
curl -s -H "Authorization: Bearer $(gcloud auth print-identity-token)" $API_URL/config
| python3 -m json.tool | grep sha256
```

EXPECTED

```
"sha256_8": "4e0d88f2"
"sha256_8": "4e0d88f2"      # both paths on version 2
```

Step 8 — Who may read it — the wrong identity is refused; you, as Owner, are not**CLOUD SHELL**

```
gcloud secrets versions access latest --secret=db-password --impersonate-service-
account=bookshelf-web-sa@$PROJECT.iam.gserviceaccount.com
gcloud secrets versions access latest --secret=db-password | wc -c
```

EXPECTED

```
ERROR: (gcloud.secrets.versions.access) PERMISSION_DENIED: Permission
'secretmanager.versions.access' denied
for resource 'projects/110919260780/secrets/db-password/versions/latest' (or it may
not exist).
17
```

You are not on the secret's policy and you can read it — Owner on the project is inherited. Lab 3's Policy Analyzer step makes that visible.

CHECK YOURSELF

- Both fingerprints matched on version 1; after rotation the file changed and the env var did not.
- The new revision with the disabled pinned version failed to start, and the service kept serving.
- bookshelf-web-sa was refused; you were not.

Symptom	Cause	Fix
Revision fails: 'Permission denied on secret'	secretAccessor not on the secret for bookshelf-api-min	step 3; wait 30 s
secret_file present:false	path typo — must be /secrets/db-password	re-run step 4 with the exact path
step 5 shows no change in secret_file	read too soon	wait 10 s more; the mount fetches on read
Cloud Build fails immediately	compute default SA lacks roles	trainer's prep-project.sh; see

Symptom	Cause	Fix
		appendix

Activity B — Boundaries for Bookshelf

Time	Format	Material
15 min + debrief	pairs, paper or text file	activity-b-template.md in the kit (printed below)

Draw the folder and project layout for Bookshelf across development, staging and production; name every service account and secret; mark ingress; then answer the blast-radius question. Fill every blank.

1 · Folders and projects

```
Organization: example.com
  folder: _____ → projects: _____ (shared registry lives here), _____
(CI)
  folder: _____ → projects: bookshelf-dev, sandboxes ← the org-policy
override (domain restricted sharing: allowAll) lives on THIS folder
  folder: _____ → projects: bookshelf-staging, bookshelf-prod ← Data Access
audit logs ON here
```

2 · Identities and secrets — six of each

Environment	bookshelf-api runs as	bookshelf-web runs as	db-password lives in	api ingress	web public?
dev	bookshelf-api-sa@_____	bookshelf-web-sa@_____	project _____, version ____		
staging					
prod					

3 · What crosses

- The only resource all three environments read: _____ (identical by digest)
- The pipeline identity _____@_____ holds roles/run.admin on projects ____, ____, ____ and roles/iam.serviceAccountUser on _____ — not on the projects

4 · Blast radius

A token for bookshelf-api-sa@bookshelf-dev leaks. List everything it can reach. If your list contains anything in staging or prod, name the shared thing that should not be shared.

```
-
-
-
shared thing that should not be shared: _____
```

Lab 3 — Least privilege, managed secret, internal ingress, two audit answers

Estimated time	Difficulty	Environment	Kit
25 min	●●●	Cloud Shell, region asia-south1	gcp-iam-labs.zip (labs.sh + bookshelf-api/ 1.1 + activity files)

OBJECTIVE

bookshelf-api on internal ingress; Data Access audit logging on for Secret Manager; one read of the secret by you and one by the service, both found in the log; Policy Analyzer listing everyone who CAN read the secret — including the inherited Owner never bound on it; and a filled answer table for an auditor. This is the state the CI/CD and Incident sessions start from — do not tear down.

PREREQUISITES

- Labs 1 and 2 complete.
- Cloud Asset API enabled (prep-project.sh does it; step 4 enables it again harmlessly).

Steps

Step 1 — The api on internal ingress

Console path: Cloud Run → bookshelf-api → Networking → Ingress control → Internal

CLOUD SHELL

```
gcloud run services update bookshelf-api --region $R --ingress internal
gcloud run services describe bookshelf-api --region $R --
format='value(metadata.annotations."run.googleapis.com/ingress")'
curl -s -o /dev/null -w "from the internet: HTTP %{http_code}\n" -H "Authorization:
Bearer $(gcloud auth print-identity-token)" $API_URL/
```

EXPECTED

```
internal
from the internet: HTTP 404
```

404, not 403: internal ingress refuses at the network edge before IAM ever sees the request. A valid token does not help. bookshelf-web still reaches it through the VPC path you set in the Cloud Run session.

Step 2 — Turn on Data Access audit logs for Secret Manager

Data Access logs are off by default. The command form edits the project policy's auditConfigs; the console path is four clicks and does the same thing.

Console path: IAM & Admin → Audit Logs → filter 'Secret Manager API' → tick Data Read and Data Write → Save

CLOUD SHELL

```
gcloud projects get-iam-policy $PROJECT --format=json > /tmp/policy.json
python3 - <<'EOF'
import json; p=json.load(open('/tmp/policy.json'))
cfg=[c for c in p.get('auditConfigs',[]) if c['service']!=
'secretmanager.googleapis.com']
cfg.append({'service':'secretmanager.googleapis.com', 'auditLogConfigs':
[{'logType':'DATA_READ'},{'logType':'DATA_WRITE'}]})
p['auditConfigs']=cfg; json.dump(p,open('/tmp/policy.json','w'),indent=1)
EOF
gcloud projects set-iam-policy $PROJECT /tmp/policy.json | grep -A4 auditConfigs
```

EXPECTED

```
auditConfigs:
- auditLogConfigs:
- logType: DATA_READ
```

```
- logType: DATA_WRITE
service: secretmanager.googleapis.com
```

The small python block only edits a JSON file on disk — it is a text edit, not application code. Use the console path if you prefer.

Step 3 — Generate two reads — one by you, one by the service

CLOUD SHELL

```
gcloud secrets versions access latest --secret=db-password >/dev/null
gcloud run services update bookshelf-api --region $R --update-env-vars TOUCH=$(date +
%s) >/dev/null
sleep 60
```

The service update starts a new instance, which fetches the env-var secret at start — that fetch is made by bookshelf-api-min and will appear in the log.

Step 4 — Who CAN read it — Policy Analyzer walks inheritance

Console path: IAM & Admin → Policy Analyzer → Custom query → Resource: the secret; Permission: secretmanager.versions.access

CLOUD SHELL

```
gcloud services enable cloudasset.googleapis.com
gcloud asset analyze-iam-policy --project=$PROJECT \
  --full-resource-name="//secretmanager.googleapis.com/projects/$PN/secrets/db-password" \
  --permissions=secretmanager.versions.access
```

EXPECTED

```
---
ACLS:
- accesses: [{permission: secretmanager.versions.access}]
  identities: [{name: 'serviceAccount:bookshelf-api-min@gcptraining-507817.iam.gserviceaccount.com'}]
  resources: [{fullResourceName:
'//secretmanager.googleapis.com/projects/110919260780/secrets/db-password'}]
policy: {attachedResource:
'//secretmanager.googleapis.com/projects/110919260780/secrets/db-password', binding:
{role: roles/secretmanager.secretAccessor}}
---
ACLS:
- accesses: [{permission: secretmanager.versions.access}]
  identities: [{name: 'user:rohan@example.com'}]
policy: {attachedResource:
'//cloudresourcemanager.googleapis.com/projects/gcptraining-507817', binding: {role:
roles/owner}}
```

Two results: the binding you made on the secret, and YOU through roles/owner attached on the project — inherited, invisible in gcloud secrets get-iam-policy. In a real org the list also expands groups and folder-level grants. The resource name uses the project NUMBER, as the audit log does. Results can lag policy changes by a few minutes.

Step 5 — Who DID read it — the Data Access log

Console path: Logging → Logs Explorer → query below

CLOUD SHELL

```
gcloud logging read 'protoPayload.serviceName="secretmanager.googleapis.com" AND
protoPayload.methodName="google.cloud.secretmanager.v1.SecretManagerService.AccessSecretVersion"' \
  --limit 10 --format='table(timestamp,
protoPayload.authenticationInfo.principalEmail, protoPayload.resourceName)'
```

EXPECTED

TIMESTAMP	PRINCIPAL_EMAIL
-----------	-----------------

```
RESOURCE_NAME
2026-09-09T10:02:41.118Z      bookshelf-api-min@gcptraining-
507817.iam.gserviceaccount.com
projects/110919260780/secrets/db-password/versions/2
2026-09-09T10:01:33.905Z      rohan@example.com
projects/110919260780/secrets/db-password/versions/latest
```

One line per read, with the caller. Reads before step 2 are not there — the log did not exist yet. That is the argument for enabling it on the prod folder on day one.

Step 6 — The answer sheet for the auditor

CLOUD SHELL

```
gcloud secrets versions list db-password
for SA in bookshelf-api-min bookshelf-web-sa; do echo "== $SA"; gcloud projects get-
iam-policy $PROJECT --flatten='bindings[].members' --filter="bindings.members:$SA@" --
format='value(bindings.role)'; done
gcloud secrets get-iam-policy db-password --format='value(bindings)'
gcloud run services get-iam-policy bookshelf-api --region $R --
format='value(bindings)'
```

The question	Your evidence (fill in)	Produced by
Who can read db-password?		Policy Analyzer (step 4)
Who did read it, when?		Data Access log (step 5)
How is it rotated?	versions: 2 enabled, 1 disabled — dates:	gcloud secrets versions list
Why does each binding exist?	api-min → secretAccessor on the secret because ... web-sa → run.invoker on the api because ...	your notes from Lab 1 step 7

CHECK YOURSELF — AND LEAVE EVERYTHING RUNNING

- Internet curl to the api returns 404; web still works.
- Two callers in the Data Access log.
- Policy Analyzer shows the inherited Owner you never bound.
- Do NOT run teardown.sh — the CI/CD and Incident sessions start here.

Symptom	Cause	Fix
Log query returns nothing	reads happened before Data Access was enabled, or < 60 s ago	repeat step 3, wait, query again
analyze-iam-policy: API not enabled	Cloud Asset API	the enable command in step 4; wait a minute
analyze-iam-policy shows only the secret binding	asset inventory lag	wait 2–3 min and re-run
web page shows 'could not reach bookshelf-api'	Direct VPC egress not set on web (Cloud Run session Lab 4)	gcloud run services update bookshelf-web --network default --subnet default --vpc-egress private-ranges-only --region \$R

Appendix — Environment, versions and the kit

Versions verified 2026-09-09

Item	Value
gcloud	583.x (Cloud Shell)
Region	asia-south1
bookshelf-api	1.1 — Flask 3.1.3, gunicorn 26.2.0, python:3.13-slim; adds GET /config
Secret Manager	v1 API; roles used: secretmanager.secretAccessor
Cloud Run secret references	--update-secrets NAME=secret:version (env) and /path=secret:version (file); env resolved at instance start, file fetched on read
Policy Analyzer	gcloud asset analyze-iam-policy; needs cloudasset.googleapis.com and roles/cloudasset.viewer (Owner has it)
Audit logs	Admin Activity always on; Data Access enabled per service in the project policy's auditConfigs

The kit

File	Purpose
labs.sh	Labs 1–3 as one command sheet — run block by block
bookshelf-api/	version 1.1, supplied complete; built with Cloud Build in Lab 2
activity-a-policy.yaml	Activity A policy dump
activity-b-template.md	Activity B template
prep-project.sh	TRAINER: per-sandbox APIs, org-policy override, Cloud Build service-account roles
teardown.sh	END OF PROGRAMME only

Trainer preparation (the day before)

Organizations created after May 2024 enforce Domain restricted sharing, which blocks allUsers; and Cloud Build in a new project runs as the compute default service account without the roles it needs. prep-project.sh fixes both per sandbox project and needs Organization Policy Administrator on the organization for the override:

CLOUD SHELL

```
bash prep-project.sh <sandbox-project-id> # repeat per project; allow a few
minutes to propagate
```

What this session leaves behind — the starting state for CI/CD and Incident Response

- bookshelf-api 1.1 on internal ingress, running as bookshelf-api-min, secret db-password mounted at /secrets and pinned as DB_PASSWORD=2
- bookshelf-web public, running as bookshelf-web-sa with run.invoker on the api
- bookshelf-deployer with run.admin and serviceAccountUser on bookshelf-api-min — the pipeline identity Session D starts from
- Data Access audit logging on for Secret Manager